Triangulation — Functional Specification

Field	Value
Document Title	Triangulation: Functional Specification
Application	Triangulation (Multi-Model Cross-Verdict Synthesizer)
Application Version	4.1
Document Version	1.0
Document Status	Released
Document Type	Functional Specification
Date	May 23, 2026

1. Introduction

1.1 Purpose

This document specifies the functional behavior, architecture, and design rationale of *Triangulation*, a single-page web application that submits a user-supplied prompt in parallel to three frontier large language models (LLMs), orchestrates a structured cross-evaluation among them, and synthesizes a single annotated response that flags claims by their evidentiary status.

The application is intended as an experimental instrument for investigating the epistemic behavior of generative AI systems — specifically, the gap between *fluency* and *veracity* in unconstrained model output. It is not a production-grade verification tool. Its value is in surfacing failure modes that inform the design of a future production-grade veracity engine.

1.2 Scope

This specification covers:

- The user-facing behavior of the application
- The orchestration logic for source generation, cross-evaluation, and synthesis
- The data model, state management, and persistence layer
- The external API integrations and their failure handling
- The user interface structure, controls, and rendering rules
- Security, performance, and known-limitation considerations

This specification does *not* cover:

- The construction of a trusted knowledge substrate (the scrubber layer); see §13 for the architectural relationship between Triangulation and that future work
- Server-side deployment, hosting, or multi-user concerns
- Compliance frameworks, accessibility audits, or formal verification

1.3 Definitions and Conventions

Term	Definition
Source response	The initial answer produced by one of the three LLMs to the user's prompt.
Judge	An LLM evaluating another LLM's source response. Each source response is judged by the two LLMs that did not produce it.
Verdict	A judge's evaluation of a source response, consisting of a numeric score (0–100) and a written explanation.
Cross-evaluation	The full matrix of six verdicts produced when each of three source responses is judged by the other two LLMs.
Synthesis	A consolidated response composed by Anthropic from all three source responses and all six verdicts, marked up with inline annotations.
Annotation	A span of text in the synthesis tagged as `<spin>` (minor concern) or `<dispute>` (contested claim).
Substrate	An external source of ground truth against which claims would be validated in a future scrubber implementation. Not present in v4.1.
Provider	One of the three integrated LLM vendors: Anthropic, OpenAI, Google.

1.4 Document Conventions

Requirements are categorized as **MUST** (mandatory), **SHOULD** (recommended), and **MAY** (optional). Requirement identifiers use the pattern `[REQ-x.y.z]` for cross-referencing.

---

2. System Overview

2.1 Product Description

Triangulation is a self-contained single-file HTML application. The entire system — user interface, orchestration logic, API clients, markdown rendering, and persistence — resides in one HTML file with embedded CSS and JavaScript. No server-side component, build pipeline, or runtime framework is required.

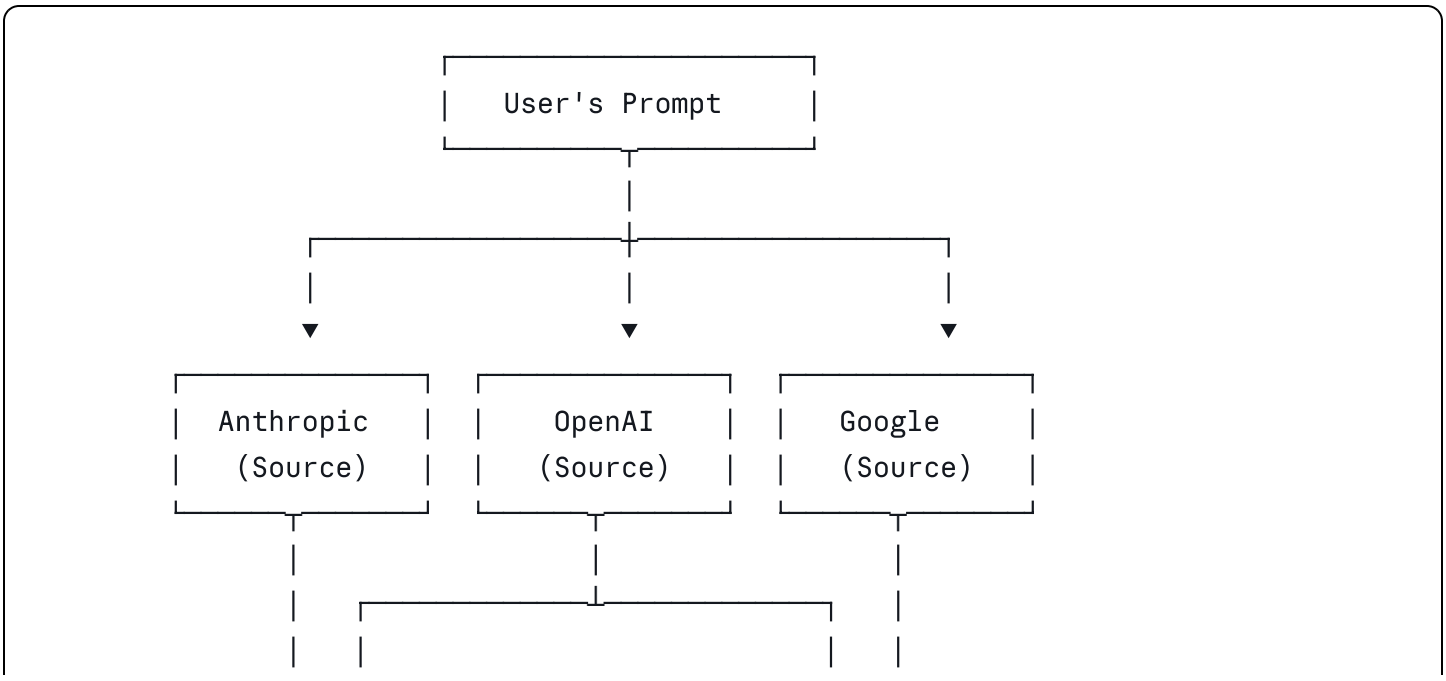
The application runs in a standard modern web browser. The user opens the file directly (via `file://` URL or local HTTP server), configures API credentials for one or more LLM providers, enters a prompt, and observes the system execute a structured cross-evaluation and synthesis workflow.

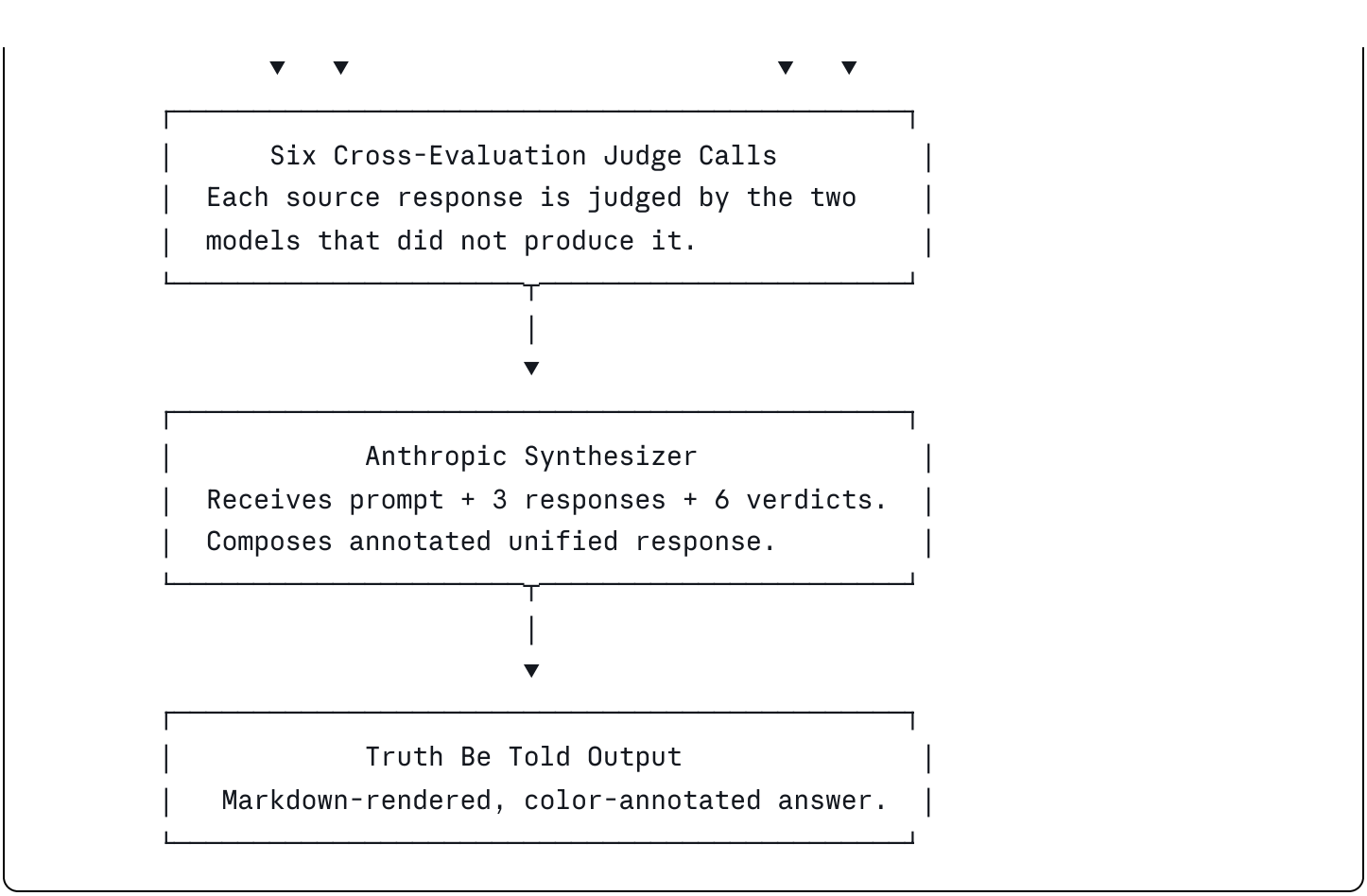
2.2 Design Philosophy

The application is designed around a single epistemic thesis: that probabilistic next-token prediction can produce highly fluent output without grounding that output in verifiable fact, and that the gap between fluency and truth is the central reliability problem of generative AI. Triangulation exists to make that gap *visible and measurable* in a structured way, so that the failure modes of LLM-based reasoning can be observed and catalogued.

The application deliberately implements peer-review-by-LLM as its evaluation mechanism, knowing that this mechanism shares the epistemic limitations of the systems it evaluates. This is a *feature*, not a bug: the application surfaces consensus hallucinations, undisclosed conflicts of interest, and temporal drift precisely because the judges and the synthesizer all have the same architectural blind spots as the source models. The instrument is calibrated to show what's broken about the paradigm it inhabits.

2.3 Conceptual Architecture





The flow comprises ten LLM API calls per user prompt: three source calls, six judge calls, and one synthesis call.

---

3. Goals and Non-Goals

3.1 Goals

- **G-1.** Allow a user to submit a single prompt and receive three independent responses from frontier LLMs in parallel.
- **G-2.** Surface the relative degree to which each model's response is considered truthful by competing models.
- **G-3.** Produce a single consolidated response that incorporates the best-supported material from all three responses, with visible flags on weakly-supported claims.
- **G-4.** Make failure modes (empty responses, parsing failures, network errors) explicitly visible rather than silent.
- **G-5.** Persist user configuration (API keys, model names) locally across sessions.
- **G-6.** Operate entirely client-side, with no server dependency, build step, or external state.

3.2 Non-Goals

- **NG-1.** Triangulation is *not* a production verification system. It does not consult any external

knowledge substrate.

- **NG-2.** Triangulation does *not* attempt to determine objective truth. It surfaces consensus, divergence, and self-reported epistemic confidence among LLMs.
- **NG-3.** Triangulation does *not* support multi-turn conversation. Each prompt is a fresh, independent run.
- **NG-4.** Triangulation does *not* manage user authentication, billing, or rate limiting. The user supplies their own API credentials and bears all API costs.
- **NG-5.** Triangulation is *not* intended for deployment on a public host. It is a local experimentation tool.

---

4. Functional Requirements

4.1 Configuration Management

[REQ-4.1.1] The application MUST provide a collapsible configuration panel containing input fields for API key and model name for each of the three providers.

[REQ-4.1.2] API key fields MUST be rendered as `password`-type inputs to obscure the credential from incidental shoulder-viewing.

[REQ-4.1.3] Configuration values MUST persist across browser sessions using `localStorage`, keyed under `triangulation_v4_1_config`.

[REQ-4.1.4] The application MUST attempt to read configuration from prior versions' storage keys as a fallback on first load. The fallback order is: `v4`, `v3.1`, `v3`, `v2`, `v1`.

[REQ-4.1.5] The application MUST provide a "Clear" action that removes all stored credentials and resets model names to default values, gated by a user confirmation prompt.

[REQ-4.1.6] The application MUST provide a "Save" action that writes the current configuration to `localStorage` and displays a transient confirmation indicator.

[REQ-4.1.7] Default model names provided at first load are:

Provider	Default Model
Anthropic	`claude-opus-4-7`
OpenAI	`gpt-5`
Google	`gemini-2.5-flash`

[REQ-4.1.8] Model names MUST be editable text fields. The user MAY enter any string the provider's API will accept.

4.2 Prompt Submission

[REQ-4.2.1] The application MUST provide a single multi-line text input for the user's prompt, with a minimum visible height of approximately four lines and a resizable handle.

[REQ-4.2.2] A primary action button MUST initiate the full workflow (source generation, cross-evaluation, synthesis).

[REQ-4.2.3] The user MUST be able to submit the prompt via keyboard shortcut: `Ctrl+Enter` (Windows/Linux) or `Cmd+Enter` (macOS).

[REQ-4.2.4] A secondary "Clear" action MUST reset all output regions to their idle state without affecting configuration.

[REQ-4.2.5] If the prompt is empty, the application MUST notify the user and decline to initiate the workflow.

[REQ-4.2.6] If no provider has a configured API key, the application MUST notify the user, open the configuration panel, and decline to initiate the workflow.

[REQ-4.2.7] While the workflow is executing, the primary action button MUST be disabled and display a "Working..." label.

4.3 Source Response Generation

[REQ-4.3.1] Upon prompt submission, the application MUST initiate one API call to each of the three providers in parallel. Calls MUST execute concurrently, not sequentially.

[REQ-4.3.2] Each provider's source call MUST use the configured model name and API key for that provider.

[REQ-4.3.3] Each source call MUST be subject to a per-provider token budget:

Provider	Source Token Budget
Anthropic	4,096 tokens
OpenAI	16,384 tokens
Google	4,096 tokens

The OpenAI budget is intentionally larger to accommodate reasoning models (e.g., GPT-5, o-series) that consume tokens for internal reasoning before producing visible output.

[REQ-4.3.4] Each source response MUST be rendered in a dedicated panel on the Responses tab, with the response text formatted using inline markdown rendering (see §4.7).

[REQ-4.3.5] Each panel MUST display the response's elapsed wall-clock time to two decimal places of seconds.

[REQ-4.3.6] Each panel MUST display a status indicator distinguishing the states: *idle*, *loading*, *success*, *error*.

[REQ-4.3.7] If a provider has no configured API key, that provider's panel MUST be set to an error state with the message "No API key configured", and the workflow MUST proceed with the remaining providers.

[REQ-4.3.8] If a source call fails for any reason (HTTP error, network error, empty response), the panel MUST display the error message, and no judge calls SHALL be initiated for that source response.

4.4 Cross-Evaluation (Judging)

[REQ-4.4.1] As soon as a source response completes successfully, the application MUST initiate two judge calls in parallel: one to each of the providers that did not produce that source response.

[REQ-4.4.2] Judge calls MUST NOT wait for other source calls to complete. The judge phase begins per-source as each source completes.

[REQ-4.4.3] Each judge call MUST receive a structured prompt containing:

- The original user prompt
- The source response being evaluated
- The identity of the source model (provider and model name)
- Instructions to produce a 0–100 score and a structured explanation citing credible sources

[REQ-4.4.4] The judge prompt MUST specify a strict output format: `SCORE: <integer>` followed by `EXPLANATION:` followed by the explanation text.

[REQ-4.4.5] The application MUST parse the judge's response to extract the numeric score and the explanation text. The parser MUST tolerate minor format deviations:

- The score MAY be on any line, in the form `SCORE: N`, `score: N`, or similar case-insensitive variations.
- If no `SCORE:` label is found, the parser MUST attempt to extract the first integer in the range 0–100 from the response.
- The explanation MAY follow an `EXPLANATION:` label, or MAY be inferred as all text following the score line.

[REQ-4.4.6] The application MUST classify each judge response into one of the following states:

State	Condition
`pending`	Judge call in flight.
`scored`	Judge returned a parseable 0–100 integer.
`unparseable`	Judge returned text but no extractable score.
`empty`	Judge returned an empty or whitespace-only content field.
`errored`	Judge call failed (network, HTTP error, etc.).

[REQ-4.4.7] Each verdict MUST be rendered in two locations:

- On the **Responses** tab, as a numeric badge attached to the source response panel, color-coded by score range.
- On the **Veracity Check** tab, as a numeric score pill plus a markdown-rendered explanation box.

[REQ-4.4.8] Score color coding MUST follow:

Score Range	Color Class
70–100	Green (high)
40–69	Amber (mid)
0–39	Red (low)

4.5 Synthesis (Truth Be Told)

[REQ-4.5.1] After all source and judge calls have settled, the application MUST initiate a single synthesis call to Anthropic.

[REQ-4.5.2] The synthesis call MUST proceed if and only if:

- At least one Anthropic API key is configured.
- At least one source response was successfully produced.

If either condition is unmet, the synthesis MUST be skipped and an explanatory error displayed in the Truth Be Told tab.

[REQ-4.5.3] The synthesis prompt MUST contain:

- The original user prompt
- All three source responses (or an explicit indication that a response is missing)
- All successful judge verdicts (score and explanation)
- Instructions for composition, including markdown formatting permissions
- Instructions for the inline annotation tags: `<spin>` for minor concerns, `<dispute>` for contested claims

[REQ-4.5.4] The synthesis call MUST use Anthropic's `claude-opus-4-7` (or the user-configured Anthropic model) with a token budget of 8,192 tokens, larger than the source call budget to accommodate the longer composed response.

[REQ-4.5.5] The synthesis response MUST be rendered in the Truth Be Told tab with full markdown rendering and the two custom annotation tags converted to colored text spans.

[REQ-4.5.6] The Truth Be Told tab MUST display:

- A legend showing the visual treatment of consensus, spin, and dispute text
- A metadata strip showing: number of source responses, number of successful evaluations, average veracity score
- The rendered synthesis text in a serif typeface
- The elapsed synthesis time

[REQ-4.5.7] While the synthesis call is in flight and the user is on a different tab, the Truth Be Told tab label MUST display a pulsing indicator dot. The indicator MUST disappear when the user selects the Truth Be Told tab.

4.6 Annotation Rendering

[REQ-4.6.1] Two custom inline tags MUST be recognized in the synthesis output and rendered as colored text spans:

Tag	Semantic	Rendered Style
`<spin>`	Minor concern, soft framing issue, or mild imprecision	Yellow text (`var(--warn)`), font-weight 500
`<dispute>`	Contested, contradicted, or stale claim, or undisclosed conflict of interest	Red text (`var(--error)`), font-weight 500

[REQ-4.6.2] Annotation spans MUST display a tooltip on hover that labels the type of concern.

[REQ-4.6.3] The annotation rendering MUST be applied via text color only. No background highlights, no underlines, no borders are permitted on annotation spans.

[REQ-4.6.4] Markdown formatting (bold, italic, code) inside annotation tags MUST be rendered correctly. For example, `<spin>**bold inside spin**</spin>` MUST render with both the color and the bold weight applied.

[REQ-4.6.5] No HTML tags other than `<spin>` and `<dispute>` from the model's output MUST reach the rendered DOM. All other HTML MUST be stripped or escaped to prevent injection attacks.

4.7 Markdown Rendering

[REQ-4.7.1] Source responses, judge explanations, and the synthesis output MUST be rendered with inline markdown processing.

[REQ-4.7.2] The markdown renderer MUST be implemented inline in the application, with no external CDN dependency.

[REQ-4.7.3] The renderer MUST support the following constructs:

- ATX-style headers (`#`, `##`, `###`, `####`)
- Bold (`**text**` and `__text__`)
- Italic (`*text*`)
- Inline code (`` `text` ``)
- Fenced code blocks (` ```lang\ncode\n``` `)
- Unordered lists (`-`, `*`, `+`)
- Ordered lists (`1.`)
- Blockquotes (`>`)
- Horizontal rules (`---`, `***`, `___`)
- Links (`[text](url)`)
- Paragraphs (separated by blank lines)
- Hard line breaks within a paragraph

[REQ-4.7.4] The renderer MUST escape all HTML in the input before applying markdown transformations. Raw HTML from the model output (other than the two annotation tags) MUST NOT pass through.

[REQ-4.7.5] Failure of the markdown renderer MUST NOT prevent the response from being displayed. The renderer MUST fall back to escaped plain text with line breaks preserved.

---

5. User Interface Specification

5.1 Layout

The application is a single-page layout with the following vertical structure:

1. Header (brand and version meta)
2. Security warning banner
3. Configuration panel (collapsible)
4. Prompt input section with action controls
5. Tab navigation (three tabs)
6. Active tab content area
7. Footer

The maximum content width is 1600 pixels. The layout is responsive: at viewport widths below 1100 pixels, the three-column grids on the Responses and Veracity Check tabs collapse to a single column.

5.2 Visual Design

The application uses a dark color palette intended to evoke a laboratory instrument or measurement device rather than a chat interface.

Token	Value	Purpose
`--bg`	`#0b0b0e`	Page background
`--bg-panel`	`#16161c`	Card / panel background
`--bg-input`	`#1c1c24`	Input field background
`--text`	`#d8d8e0`	Primary text
`--text-dim`	`#8a8a95`	Secondary text
`--text-faint`	`#555560`	Tertiary text and labels
`--accent`	`#d4a574`	Primary action color (amber)
`--anthropic`	`#d68a5a`	Anthropic provider color
`--openai`	`#7bc99a`	OpenAI provider color
`--google`	`#7ba4d6`	Google provider color
`--warn`	`#d4b96a`	Annotation: spin
`--error`	`#d67a7a`	Annotation: dispute
`--success`	`#7bc99a`	Successful state

The typography pairs `JetBrains Mono` (monospace, for instrumentation and data) with `IBM Plex Serif` (italic, for human-readable headings and the synthesis output). The serif/italic treatment for the synthesis is intentional: it signals the deliverable nature of that output, distinct from the structured measurement data on other tabs.

5.3 Tab Structure

5.3.1 Responses Tab

Three columns side-by-side, one per provider. Each column contains:

- Provider header (name and model)
- Veracity verdicts band (two compact score badges from the judging models)
- Status indicator and elapsed time
- Response body (markdown-rendered)

The veracity band sits *above* the response body so that the user sees the judgment summary before reading the response.

5.3.2 Veracity Check Tab

Three columns side-by-side, one per source response. Each column contains:

- Subject header identifying the response being judged
- Two judge blocks, each containing:
  - Judge identity
  - Score pill (color-coded)
  - Explanation box (markdown-rendered, scrollable, vertically resizable)

5.3.3 Truth Be Told Tab

A single full-width container with:

- Title and subtitle
- Status indicator with elapsed time
- Legend showing colored text examples (consensus, spin, dispute)
- Metadata strip (sources, evaluations, average veracity score)
- Synthesis body (serif typeface, markdown-rendered, color-annotated)

5.4 Status Indicators

A small colored dot conveys panel state across the application:

Color	Animation	Meaning
Gray (faint)	None	Idle
Amber	Pulse (1s cycle)	Loading
Green	None	Success
Red	None	Error
Amber (verdict only)	None	Empty response (judge produced no content)

5.5 Accessibility Considerations

The application does not provide a formal accessibility audit. The following deliberate choices have been made:

- All interactive elements are reachable by keyboard tab order.
- Color is supplemented by text labels and shape (status text plus colored dot, score numerals plus colored borders) so that information is not conveyed by color alone.
- The dark color scheme is selected for legibility on standard displays. A light-mode variant is not implemented.

---

6. Data Model

6.1 Persistent State

The only persistent state is the configuration object stored in `localStorage` under the key `triangulation_v4_1_config`:

```javascript
{
  anthropicKey:   string,   // API key, persisted in cleartext
  openaiKey:      string,
  googleKey:      string,
  anthropicModel: string,   // model identifier
  openaiModel:    string,
  googleModel:    string
}
```

6.2 Transient Run State

A single in-memory object holds the data of the current run. It is reset to empty on each prompt submission.

```javascript
```

```
runState = {
  prompt: string,
  responses: {
    anthropic: string | null,
    openai:    string | null,
    google:    string | null
  },
  judgments: {
    "anthropic-openai":  { score: number, explanation: string },
    "anthropic-google":  { score: number, explanation: string },
    "openai-anthropic":  { score: number, explanation: string },
    "openai-google":     { score: number, explanation: string },
    "google-anthropic":  { score: number, explanation: string },
    "google-openai":     { score: number, explanation: string }
  }
}
```

Judgment keys use the convention `{source}-{judge}`, where `source` is the provider whose response is being judged and `judge` is the provider doing the judging.

The synthesis call reads from this object after all source and judge calls have settled.

---

7. External Interfaces

7.1 Anthropic Messages API

- **Endpoint:** `https://api.anthropic.com/v1/messages`
- **Method:** `POST`
- **Required Headers:**
  - `x-api-key: <key>`
  - `anthropic-version: 2023-06-01`
  - `anthropic-dangerous-direct-browser-access: true`
  - `content-type: application/json`
- **Request Body:**

```
json
```

```
{
  "model": "<model-id>",
  "max_tokens": <int>,
  "messages": [{ "role": "user", "content": "<prompt>" }]
}
```

- **Response Parsing:** Concatenate the `text` fields from each item in `data.content`.
- **Empty-Response Detection:** If the concatenated text is empty or whitespace-only, raise `EmptyResponseError` with `stop_reason` from the response.

The `anthropic-dangerous-direct-browser-access: true` header is required by Anthropic's API to permit credential exposure in client-side code. The application's security warning banner reflects this risk.

7.2 OpenAI Chat Completions API

- **Endpoint:** `https://api.openai.com/v1/chat/completions`
- **Method:** `POST`
- **Required Headers:**
  - `Authorization: Bearer <key>`
  - `content-type: application/json`
- **Request Body:**

json

```
{
  "model": "<model-id>",
  "messages": [{ "role": "user", "content": "<prompt>" }],
  "max_completion_tokens": <int>
}
```

- **Response Parsing:** `data.choices[0].message.content`
- **Empty-Response Detection:** If `content` is null or whitespace-only, raise `EmptyResponseError` annotated with `finish_reason` and token usage including reasoning tokens if available.

The `max_completion_tokens` parameter (as opposed to the legacy `max_tokens`) is required for reasoning-capable models. The 16K budget specifically accommodates GPT-5 and o-series models that consume reasoning tokens against this budget before producing visible output.

7.3 Google Gemini API

- **Endpoint:** `https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent?key={key}`
- **Method:** `POST`
- **Required Headers:** `content-type: application/json`
- **Request Body:**

```
json

  {
    "contents": [{ "parts": [{ "text": "<prompt>" }] }],
    "generationConfig": { "maxOutputTokens": <int> }
  }
```

- **Response Parsing:** Concatenate the `text` fields from `data.candidates[0].content.parts`.
- **Empty-Response Detection:** If the concatenated text is empty, raise `EmptyResponseError` annotated with `finishReason`.

7.4 Common Error Handling

All three provider client functions follow the same error pattern:

- HTTP non-2xx responses raise a generic `Error` with the status code and response body.
- Empty content responses raise the custom `EmptyResponseError` with provider-specific diagnostic information.
- All other failures (network, JSON parse, etc.) propagate as standard JavaScript errors.

---

8. Workflow and Use Cases

8.1 Primary Use Case: Submit Prompt and Receive Synthesized Answer

Actor: Researcher exploring LLM epistemic behavior.

Preconditions: At least one API key is configured. The Anthropic key is configured (required for synthesis).

Flow:

1. User types a prompt and submits via button click or `Ctrl+Enter`.
2. Application resets all output regions and the run state.

3. Application initiates three parallel source calls.
4. As each source call completes:
    - If successful: response is displayed in the corresponding panel; two judge calls are initiated in parallel.
    - If failed: error is displayed in the corresponding panel; the two affected judge verdicts are marked as `errored`.
5. Judge calls complete asynchronously; verdicts populate on both the Responses and Veracity Check tabs as they arrive.
6. After all source and judge calls have settled (via `Promise.allSettled`), the application initiates the synthesis call.
7. While synthesis is in flight, the Truth Be Told tab label pulses (if the user is on another tab).
8. Synthesis response is rendered in the Truth Be Told tab with markdown formatting and inline annotations.
9. The action button is re-enabled.

Postcondition: All three tabs reflect the results of the run. The user may switch tabs to examine raw responses, detailed cross-evaluations, or the synthesized answer.

8.2 Secondary Use Cases

8.2.1 Configure credentials. User opens the Configuration panel, enters or modifies keys and model names, clicks Save. Configuration is persisted to localStorage.

8.2.2 Clear stored credentials. User clicks Clear, confirms the action, and all stored configuration is removed.

8.2.3 Clear outputs without resubmitting. User clicks the Clear (responses) action; all panels return to idle state; configuration is unaffected.

8.2.4 Partial provider availability. User has only one or two providers configured. Workflow proceeds with available providers; missing providers' panels show "No API key configured" and any verdicts that would have involved a missing provider are marked errored.

---

9. Failure States and Recovery

9.1 Source-Call Failures

Failure Mode	Detection	User-Visible Behavior
No API key	Missing configuration	Panel displays "No API key configured"; no judge calls initiated for this source.

Failure Mode	Detection	User-Visible Behavior
HTTP error	Non-2xx response	Panel displays HTTP status code and response body; no judge calls initiated.
Empty response	Content field empty or whitespace	Panel displays empty-response error with provider-specific diagnostic (finish reason, token usage).
Network failure	Fetch rejection	Panel displays the network error message.

9.2 Judge-Call Failures

State	Detection	UI Treatment
`errored`	API or network error	Score badge: `—` (gray). Explanation: error reason.
`empty`	Empty content with 2xx status	Score badge: `EMPTY` (amber). Explanation: diagnostic note about token-budget exhaustion.
`unparseable`	Non-empty content but no extractable integer	Score badge: `?` (gray). Explanation: raw response text shown.

The distinction between these three states is intentional. An earlier version of the application conflated them into a single "failed" state, which obscured the diagnostic value of failure data. Each state has a different remediation: `errored` may need credential or network attention; `empty` typically indicates token-budget issues with reasoning models; `unparseable` indicates a prompt-format problem.

9.3 Synthesis Failures

The synthesis call can fail for any of the reasons a normal Anthropic call can fail. The Truth Be Told tab displays the failure reason in the synthesis body. Additionally:

- If no Anthropic key is configured, the synthesis is skipped with a "No Anthropic key" message.
- If no source responses succeeded, the synthesis is skipped with a "No data" message.

9.4 JavaScript Errors

A defensive coding pattern is applied throughout: object lookups that may return undefined use optional chaining; string operations that may receive undefined values guard with `|| ''`; the parser always returns a fully populated result object regardless of input pathology. Earlier

development surfaced a class of bugs in which a downstream consumer crashed on an upstream's incomplete result; v3.1 introduced this defensive pattern and v4.x retains it.

---

10. Security Considerations

10.1 Credential Storage

API keys are stored in browser `localStorage` in plaintext. This is acceptable for the application's intended use (local experimentation on the user's own machine) but is *not* acceptable for any deployment scenario in which the HTML file or the user's browser session is exposed to untrusted parties.

A persistent banner at the top of the application warns the user not to deploy the page publicly.

10.2 Credential Transmission

API keys are transmitted directly from the browser to each provider's API endpoint over HTTPS. The keys are not transmitted to any other party. The application has no server-side component that could intercept credentials.

The user should be aware that:

- API keys appear in the browser's network inspector and developer tools.
- API calls bypass any rate limiting, logging, or compliance controls that a server-side proxy would normally provide.
- Browser extensions with sufficient privileges may inspect or exfiltrate credentials.

10.3 Output Sanitization

Model output is rendered as HTML in three places: source response panels, judge explanation boxes, and the synthesis body. To prevent injection of malicious markup, the application implements the following pipeline:

1. For source responses and judge explanations: all HTML tags in the input are stripped before markdown rendering. The only HTML in the rendered DOM is HTML produced by the local markdown renderer.
2. For the synthesis: `<spin>` and `<dispute>` tags are extracted to placeholders before HTML stripping; all other HTML is stripped; markdown is rendered; the two preserved tags are restored as `<span>` elements with constrained class names and text-only `title` attributes.

This pipeline is sufficient against accidental or adversarial HTML from model output. It does not constitute a formal sanitizer audit.

10.4 Cross-Origin Considerations

The application is delivered as a single static file. It does not make requests to its own origin. The three external API endpoints all permit cross-origin requests under the conditions specified:

- Anthropic requires the `anthropic-dangerous-direct-browser-access: true` header. This header is set unconditionally.
- OpenAI permits browser requests without additional configuration.
- Google permits browser requests when the API key is passed in the query string.

---

11. Performance Characteristics

11.1 Latency Profile

Each user prompt initiates ten LLM API calls. End-to-end latency is dominated by the slowest call in each phase.

- **Phase 1 (source calls):** Latency equals the maximum of three parallel calls. Typical range: 5–40 seconds, with reasoning models (GPT-5) often at the upper end.
- **Phase 2 (judge calls):** Each source response triggers two parallel judge calls. Total Phase 2 latency equals the maximum of six parallel calls (each waiting only for its source's completion). Typical range: 10–40 seconds.
- **Phase 3 (synthesis):** A single Anthropic call processing the full bundle of prior outputs. Typical range: 15–60 seconds.

Total wall-clock time per prompt: typically 30–120 seconds.

11.2 Token Consumption

A representative run consumes approximately:

- 3 × (prompt tokens + source response tokens) for the three source calls
- 6 × (prompt tokens + source response tokens + judge prompt overhead + judge response tokens) for the six judge calls
- 1 × (prompt + three responses + six evaluations + synthesis instructions + synthesis response) for the synthesis call

The synthesis call is the largest single input and may approach context-window limits for prompts that generate verbose source responses and verbose evaluations. The 8K-token output budget is generous to accommodate detailed synthesis.

11.3 Cost

Costs are borne entirely by the user via their three provider accounts. A representative single-prompt run costs on the order of $0.05–$0.50 USD depending on the prompt length, response verbosity, and the specific models selected. Reasoning models (GPT-5, o-series) are typically the dominant cost driver due to internal reasoning tokens that consume budget even when not directly visible in the output.

---

12. Known Limitations

12.1 Architectural Limitations

L-1. No external knowledge substrate. Triangulation evaluates LLM output using other LLMs. The judges and the synthesizer share the same architectural blind spots as the source models. Consensus among models does not establish truth; three models can confidently agree on a falsehood that none of them would have flagged in isolation.

L-2. Synthesizer self-reference. Anthropic produces both one of the three source responses *and* the synthesis. The synthesizer is therefore both player and referee. This creates a structural conflict of interest that is not mitigated in v4.1. The user should be aware that the synthesis may exhibit self-favorable bias when evaluating its own source response.

L-3. Cited sources are not verified. Judges are instructed to cite credible sources in their explanations. The application does not verify that these citations exist, are accurately summarized, or actually support the claims they are attached to. Citation hallucination is a known failure mode of LLMs and will appear in judge explanations.

L-4. Annotation accuracy is unverified. The synthesizer applies `<spin>` and `<dispute>` tags based on its own assessment of judge feedback. The application does not validate that the right claims were tagged or that the tagging policy was applied consistently.

12.2 Implementation Limitations

L-5. Single-prompt, no history. Each run is independent. The application does not support multi-turn conversation or accumulation of context across prompts.

L-6. No retry logic. Failed calls are not automatically retried. The user must re-submit the prompt to retry a failed run.

L-7. No request cancellation. Once a run is initiated, in-flight calls cannot be cancelled. Closing the page or navigating away will terminate the calls but may still incur token charges for work the provider has already started.

L-8. No export of results. The user cannot save, export, or share a run's output other than by copy-pasting from the rendered panels or by using the browser's save-page functionality.

L-9. Hard-coded provider set. Only Anthropic, OpenAI, and Google are supported. Adding a fourth provider, swapping providers, or using a self-hosted model requires code modification.

12.3 User Experience Limitations

L-10. No streaming output. Responses are displayed only after the full response has been received. There is no token-by-token streaming visualization.

L-11. Markdown renderer is a subset. The inline renderer supports the most common markdown features but is not a full CommonMark or GFM implementation. Edge cases (nested code spans, mixed list types, table syntax beyond the basic form) may render imperfectly.

---

13. Future Roadmap

This section is non-normative and describes anticipated successors to v4.1.

13.1 Substrate-Grounded Verification (v5+)

The principal limitation of v4.1 is L-1: LLM-on-LLM evaluation. The next-generation architecture introduces an external knowledge substrate against which claims are validated.

The substrate may take one of several forms:

- **Tool-grounded:** Specific claim categories are routed to deterministic tools (calculator, code interpreter, web search with structured extraction).
- **Document-grounded:** A user-supplied document corpus is indexed; claims are validated against retrieval from this corpus with explicit citation to source passages.
- **Knowledge-graph-grounded:** Structured facts (entity-relationship triples with provenance) form the validation layer.
- **Hybrid:** A policy engine routes different claim types to different substrates.

In each case, the verifier's role shifts from "assess on prior knowledge" (v4.1 judges) to "validate against a specific external source" (v5+ scrubber).

13.2 Anticipated v5 Feature: Substrate Tab

A fourth tab is the most likely first step toward substrate integration. The user pastes an authoritative passage into a text area; the synthesis is re-evaluated against this passage; claims are re-tagged with a four-state coloring: supported (green), unmarked (gray), uncovered (yellow), contradicted (red).

This minimal substrate is sufficient to demonstrate the architectural shift while remaining fully client-side and prompt-driven.

13.3 Anticipated v6 Feature: Automated Retrieval

Replaces user-pasted substrate with automated retrieval from a configured corpus. At this point the application begins to overlap meaningfully with Webspinner-class local-RAG systems and ceases to be a pure experimentation tool.

13.4 Other Possible Extensions

- Streaming response visualization
- Run history and export
- Configurable provider list and per-provider parameter tuning
- Pluggable synthesizer selection (rotate which model performs synthesis)
- A/B comparison of multiple synthesis strategies on the same source data
- Batch mode for running the same prompt with varied parameters
- Persistent run log (with explicit user opt-in given the credentials-and-data sensitivity)

---

14. Document Control

14.1 Revision History

Version	Date	Author	Summary
1.0	2026-05-23	(initial author)	Initial release covering application version 4.1.

14.2 Related Documents

- Application source: `triangulation_v4_1.html`
- Predecessor versions: `triangulation_v1.html` through `triangulation_v4.html` (incremental development history)

14.3 Acknowledgments

This document specifies a system designed and built as an instrument for investigating epistemic failure modes in generative AI. The application's intentional limitations are themselves part of its design: it surfaces what does not work in current LLM-based reasoning so that the architecture of a successor system — one grounded in an external knowledge substrate — can be designed with full awareness of what that successor must accomplish.
